```python
from django.db import models
from django.contrib.auth.models import AbstractUser
from django.utils.translation import gettext_lazy as _


# ── Geographic / Organizational models ──────────────────────────────


class Governorate(models.Model):
    name = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.name


class SchoolEstablishment(models.Model):
    name = models.CharField(max_length=200)
    governorate = models.ForeignKey(Governorate,
on_delete=models.CASCADE, related_name="establishments")

    def __str__(self):
        return f"{self.name} ({self.governorate.name})"


class SchoolClass(models.Model):
    name = models.CharField(max_length=100)
    establishment = models.ForeignKey(SchoolEstablishment,
on_delete=models.CASCADE, related_name="classes")

    def __str__(self):
        return f"{self.name} - {self.establishment.name}"


# ── Custom User ──────────────────────────────────────────────────────


class User(AbstractUser):
    class Role(models.TextChoices):
        SUPER_ADMIN = "SUPER_ADMIN", _("Super Admin")
        GLOBAL_ADMIN = "GLOBAL_ADMIN", _("Global Admin")
        OPERATOR = "OPERATOR", _("Operator")
        REGIONAL_ADMIN = "REGIONAL_ADMIN", _("Regional Admin")
        PRACTITIONER = "PRACTITIONER", _("Practitioner")
```

```
    class Status(models.TextChoices):
        PENDING = "PENDING", _("Pending")
        ACTIVE = "ACTIVE", _("Active")
        DISABLED = "DISABLED", _("Disabled")

    class OrgType(models.TextChoices):
        NATIONAL = "NATIONAL", _("National")
        REGIONAL = "REGIONAL", _("Regional")
        LOCAL = "LOCAL", _("Local")

    email = models.EmailField(_("email address"), unique=True)
    role = models.CharField(max_length=20, choices=Role.choices,
default=Role.PRACTITIONER)
    status = models.CharField(max_length=20, choices=Status.choices,
default=Status.PENDING)
    organization_type = models.CharField(max_length=20,
choices=OrgType.choices, default=OrgType.LOCAL)

    establishment = models.ForeignKey(SchoolEstablishment,
on_delete=models.SET_NULL, null=True, blank=True)
    governorate = models.ForeignKey(Governorate,
on_delete=models.SET_NULL, null=True, blank=True)
    phone_number = models.CharField(max_length=20, blank=True,
null=True)

    # Invitation fields
    invite_token = models.CharField(max_length=100, blank=True,
null=True)
    token_expiration = models.DateTimeField(blank=True, null=True)
    created_by = models.ForeignKey('self', on_delete=models.SET_NULL,
null=True, blank=True, related_name="invited_users")

    # Security fields
    failed_attempts = models.IntegerField(default=0)
    last_login_attempt = models.DateTimeField(null=True, blank=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']

    def __str__(self):
        return f"{self.username} ({self.role}) - {self.status}"
```

```python
class AuditLog(models.Model):
    user = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True)
    action = models.CharField(max_length=255)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.timestamp} | {self.user} | {self.action}"


class PlatformTerminology(models.Model):
    key = models.CharField(max_length=255, unique=True)
    value_fr = models.TextField()
    value_ar = models.TextField(blank=True, null=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.key} -> {self.value_fr[:30]}..."


# ——— Central Anchor
————————————————————————————————————————


class QuestionnaireSession(models.Model):
    class Language(models.TextChoices):
        FR = "FR", "French"
        AR = "AR", "Arabic"

    school_class = models.ForeignKey(SchoolClass,
on_delete=models.CASCADE, related_name="sessions", null=True,
blank=True)
    school = models.ForeignKey(SchoolEstablishment,
on_delete=models.CASCADE, related_name="sessions", null=True,
blank=True)
    governorate = models.ForeignKey(Governorate,
on_delete=models.CASCADE, related_name="sessions", null=True,
blank=True)
    language_used = models.CharField(max_length=2,
choices=Language.choices, default=Language.FR)
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    # Tethering to Class Report
    class_report = models.ForeignKey(
        'ClassReport',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="submissions",
        help_text="Link to the administrative report for this session"
    )

    # Computed risk flags for fast dashboard queries
    tobacco_user = models.BooleanField(default=False)
    ecig_user = models.BooleanField(default=False)
    hookah_user = models.BooleanField(default=False)
    alcohol_user = models.BooleanField(default=False)
    tranquilizer_user = models.BooleanField(default=False)
    cannabis_user = models.BooleanField(default=False)
    cocaine_user = models.BooleanField(default=False)
    ecstasy_user = models.BooleanField(default=False)
    heroin_user = models.BooleanField(default=False)
    inhalant_user = models.BooleanField(default=False)
    has_risk_behavior = models.BooleanField(default=False)

    # Validation / Exclusion flags
    is_valid = models.BooleanField(default=True)
    exclusion_reason = models.TextField(blank=True, null=True)

    # Dynamic Data Storage
    extra_answers = models.JSONField(default=dict, blank=True,
help_text="Stores answers for dynamically added questions")

    def __str__(self):
        return f"Session {self.id} | {self.school} |
{self.created_at.date()}"


# ── CHOICE CONSTANTS ──────────────────────────────────────────

FREQUENCY_LIFETIME = [
    ("1", "Jamais"),
    ("2", "1-2 fois"),
    ("3", "3-5 fois"),
```

```
    ("4", "6-9 fois"),
    ("5", "10-19 fois"),
    ("6", "20-39 fois"),
    ("7", ">=40 "),
]

FREQUENCY_30DAYS_CIGS = [
    ("1", "Jamais"),
    ("2", "< 1 cigarette par jour"),
    ("3", "1-5 cigarettes par jour"),
    ("4", "6-10 cigarettes par jour"),
    ("5", "11-20 cigarettes par jour"),
    ("6", "> 20 cigarettes par jour"),
]

FREQUENCY_30DAYS_VAPE = [
    ("1", "Jamais"),
    ("2", "< 1 fois par semaine"),
    ("3", "Au moins 1 fois par semaine"),
    ("4", "Tous les jours ou presque tous les jours"),
]

FREQUENCY_30DAYS_HOOKAH = [
    ("1", "Jamais"),
    ("2", "< 1 par semaine"),
    ("3", "< 1 par jour"),
    ("4", "1-5 par jour"),
    ("5", "6-10 par jour"),
    ("6", "11-20 par jour"),
    ("7", "> 20 par jour"),
]

FREQUENCY_ACTIVITIES = [
    ("1", "Jamais"),
    ("2", "Quelques fois par an"),
    ("3", "1 ou 2 fois par mois"),
    ("4", "Au moins une fois par semaine"),
    ("5", "Tous les jours"),
]

FREQUENCY_DIGITAL = [
    ("1", "Aucune"),
    ("2", "Une demi-heure ou moins"),
```

```
    ("3", "Environ 1 heure"),
    ("4", "Environ 2-3 heures"),
    ("5", "Environ 4-5 heures"),
    ("6", "6 heures ou plus"),
]

FREQ_GAMBLING_RETURN = [
    ("1", "Chaque fois"),
    ("2", "La plupart du temps"),
    ("3", "Quelques fois"),
    ("4", "Jamais"),
]

CT06A_FREQUENCY = [
    ("1", "Chaque fois"),
    ("2", "La plupart du temps"),
    ("3", "Quelques fois"),
    ("4", "Jamais"),
]

CT06_YES_NO = [
    ("1", "Oui"),
    ("2", "Non"),
]

FREQ_GAMBLING_GRID = [
    ("1", "Je n'ai pas joué"),
    ("2", "1 fois par mois ou moins"),
    ("3", "2-4 fois par mois"),
    ("4", "2-3 fois par semaine"),
    ("5", "4-5 fois par semaine"),
    ("6", "6 fois par semaine ou plus"),
]

AGE_SCALE = [
        ("1", "Jamais"),
        ("2", "<=9 ans"),
        ("3", "10 ans"),
        ("4", "11 ans"),
        ("5", "12 ans"),
        ("6", "13 ans"),
        ("7", "14 ans"),
        ("8", "15 ans"),
```

```
        ("9", ">=16 ans"),
]

DIFFICULTY_LEVELS = [
    ("1", "Impossible"),
    ("2", "Difficile"),
    ("3", "Facile"),
    ("4", "Ne sait pas"),
]

SATISFACTION_SCALE = [
    ("1", "Très satisfait(e)"),
    ("2", "Satisfait(e)"),
    ("3", "Ni satisfait(e) ni insatisfait(e)"),
    ("4", "Pas tellement satisfait(e)"),
    ("5", "Non satisfait(e)"),
    ("6", "Non applicable"),
]

AGREEMENT_SCALE_PAPER = [
    ("1", "Tout à fait d'accord"),
    ("2", "Partiellement d'accord"),
    ("3", "Ni d'accord ni pas d'accord"),
    ("4", "Partiellement en désaccord"),
    ("5", "Pas du tout d'accord"),
]

YES_NO = [("1", "Oui"), ("2", "Non")]
YES_NO_DK = [("1", "Oui"), ("2", "Non"), ("3", "Ne sait pas")]

STRESS_FREQ = [
    ("1", "Jamais"),
    ("2", "Presque jamais"),
    ("3", "Parfois"),
    ("4", "Assez souvent"),
    ("5", "Très souvent"),
]

HONESTY_SCALE = [
    ("1", "J’ai déjà déclaré [cela]"),
    ("2", "Oui, sans aucun doute"),
    ("3", "Probablement oui"),
    ("4", "Probablement pas"),
```

```
    ("5", "Non, sans aucun doute"),
]


# ── Section A — Informations Générales ──────────────────────────────

class SectionA(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_a")

    gender = models.JSONField(default=list, blank=True)
    birth_month = models.PositiveSmallIntegerField(null=True,
blank=True)
    birth_year = models.PositiveSmallIntegerField(null=True,
blank=True)

    activities_frequency = models.JSONField(default=dict, blank=True)

    academic_performance = models.JSONField(default=list, blank=True)

    household_members = models.JSONField(default=list, blank=True)

    parents_absence_reason = models.JSONField(default=list, blank=True)

    nights_out_30days = models.JSONField(default=list, blank=True)

    family_relationship_satisfaction = models.JSONField(default=dict,
blank=True)
    school_absences = models.JSONField(default=dict, blank=True)
    school_appreciation = models.JSONField(default=list, blank=True)
    parents_absence_reason_other = models.CharField(max_length=255,
blank=True,null=True)

    def __str__(self):
        return f"SectionA | Session {self.session_id}"


# ── Section B — Famille et Situation Socio-Economique ──────────────

class SectionB(models.Model):
```

```
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_b")

    EDUCATION_LEVEL = [
        ("1", "Non scolarisé"),
        ("2", "Niveau primaire"),
        ("4", "Niveau collège"),
        ("6", "Niveau secondaire"),
        ("8", "Niveau universitaire ou plus"),
        ("9", "Ne sait pas"),
        ("10", "Non applicable"),
        ("11", "Formation professionnelle"),
    ]

    father_education = models.JSONField(default=list, blank=True)
    mother_education = models.JSONField(default=list, blank=True)

    # Sous-questions CB01
    father_primary_success = models.JSONField(default=list, blank=True)
    father_middle_success = models.JSONField(default=list, blank=True)
    father_secondary_success = models.JSONField(default=list,
blank=True)

    # Sous-questions CB02
    mother_primary_success = models.JSONField(default=list, blank=True)
    mother_middle_success = models.JSONField(default=list, blank=True)
    mother_secondary_success = models.JSONField(default=list,
blank=True)

    EMPLOYMENT_STATUS = [
        ("1", "Oui, à plein temps"),
        ("2", "Oui, à temps partiel"),
        ("3", "Ne travaille pas"),
        ("4", "Je ne sais pas"),
        ("5", "Non applicable"),
        ("6", "Retraité(e)"),
    ]

    father_job = models.JSONField(default=list, blank=True)
    mother_job = models.JSONField(default=list, blank=True)

    economic_status = models.JSONField(default=list, blank=True)
    private_room = models.JSONField(default=list, blank=True)
```

```
    def __str__(self):
        return f"SectionB | Session {self.session_id}"


# ——— Sections C à M ———————————————————————————————————

class SectionC(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_c")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    age_daily_use = models.JSONField(default=list, blank=True)

class SectionD(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_d")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    age_daily_use = models.JSONField(default=list, blank=True)

class SectionE(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_e")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    age_daily_use = models.JSONField(default=list, blank=True)

class SectionG(models.Model):
```

```
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_g")
    access_difficulty = models.JSONField(default=dict, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    days_30_by_type = models.JSONField(default=dict, blank=True)
    binge_drinking_30days = models.JSONField(default=list, blank=True)
    intoxication_lifetime = models.JSONField(default=list, blank=True)
    intoxication_12months = models.JSONField(default=list, blank=True)
    intoxication_30days = models.JSONField(default=list, blank=True)
    age_first_drink = models.JSONField(default=list, blank=True)
    age_first_intoxication = models.JSONField(default=list, blank=True)

class SectionH(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_h")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)

class SectionI(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_i")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    use_last_12months = models.JSONField(default=list, blank=True)
    cannabis_types_12months = models.JSONField(default=dict,
blank=True)
    cannabis_problems_12months = models.JSONField(default=dict,
blank=True)

class SectionJ(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_j")
```

```
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    use_last_12months = models.JSONField(default=list, blank=True)

class SectionK(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_k")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    use_last_12months = models.JSONField(default=list, blank=True)

class SectionL(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_l")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    use_last_12months = models.JSONField(default=list, blank=True)

class SectionM(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_m")
    access_difficulty = models.JSONField(default=list, blank=True)
    social_circle = models.JSONField(default=dict, blank=True)
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    days_30_freq = models.JSONField(default=list, blank=True)
    age_first_use = models.JSONField(default=list, blank=True)
    use_last_12months = models.JSONField(default=list, blank=True)
```

```python
# ── Section N ──────────────────────────────────────────────────────────

class SectionN(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_n")
    access_difficulty = models.JSONField(default=dict, blank=True)
    lifetime_freq_by_type = models.JSONField(default=dict, blank=True)
    months_12_freq_by_type = models.JSONField(default=dict, blank=True)
    age_first_use_by_type = models.JSONField(default=dict, blank=True)


# ── Section P ──────────────────────────────────────────────────────────

class SectionP(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_p")
    lifetime_freq = models.JSONField(default=list, blank=True)
    months_12_freq = models.JSONField(default=list, blank=True)
    forms = models.JSONField(default=dict, blank=True)
    substances = models.JSONField(default=dict, blank=True)


# ── Section Q ──────────────────────────────────────────────────────────

class SectionQ(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_q")
    risk_perceptions = models.JSONField(default=dict, blank=True)
    help_sources = models.JSONField(default=dict, blank=True)
    friend_use_risk = models.JSONField(default=list, blank=True)


# ── Section R ──────────────────────────────────────────────────────────

class SectionR(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_r")
    hours_per_day_breakdown = models.JSONField(default=dict,
blank=True)
```

```
    agreement = models.JSONField(default=dict, blank=True)


# ── Section S ──────────────────────────────────────────────────────────

class SectionS(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_s")
    hours_per_day_school = models.JSONField(default=list, blank=True)
    hours_per_day_non_school = models.JSONField(default=list,
blank=True)
    days_per_week = models.JSONField(default=list, blank=True)
    agreement = models.JSONField(default=dict, blank=True)


# ── Section T ──────────────────────────────────────────────────────────

class SectionT(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_t")
    months_12_freq = models.JSONField(default=list, blank=True)
    offline_games = models.JSONField(default=dict, blank=True)
    online_games = models.JSONField(default=dict, blank=True)
    felt_need_increase = models.JSONField(default=list, blank=True)
    lied_about_it = models.JSONField(default=list, blank=True)
    gambling_problems = models.JSONField(default=dict, blank=True)


# ── Section U ──────────────────────────────────────────────────────────

class SectionU(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_u")
    fights_12months = models.JSONField(default=list, blank=True)
    fight_circumstances = models.JSONField(default=list, blank=True)
    fight_location = models.JSONField(default=list, blank=True)
    staff_intervention = models.JSONField(default=list, blank=True)
    fight_consequences = models.JSONField(default=list, blank=True)
    serious_injury_12months = models.JSONField(default=list,
blank=True)
```

```
    harassment = models.JSONField(default=dict, blank=True)


# ── Section V ──────────────────────────────────────────────────────────

class SectionV(models.Model):
    session = models.OneToOneField(QuestionnaireSession,
on_delete=models.CASCADE, related_name="section_v")
    a = models.JSONField(default=list, blank=True)
    b = models.JSONField(default=list, blank=True)
    c = models.JSONField(default=list, blank=True)
    d = models.JSONField(default=list, blank=True)
    self_esteem = models.JSONField(default=dict, blank=True)
    mental_health_states = models.JSONField(default=dict, blank=True)
    help_seeking = models.JSONField(default=dict, blank=True)
    stress_metrics = models.JSONField(default=dict, blank=True)


# ── Section Z ──────────────────────────────────────────────────────────

class SectionZ(models.Model):
    """Section Z du questionnaire MedSPAD (C.Z01 et C.Z02)."""

    session = models.OneToOneField(
        QuestionnaireSession,
        on_delete=models.CASCADE,
        related_name="section_z",
    )

    # C.Z01 — honnêteté concernant la consommation d'alcool
    honesty_level = models.CharField(max_length=2, blank=True)

    # C.Z02 — honnêteté concernant la consommation de cannabis
    honesty_cannabis = models.CharField(max_length=2, blank=True)

    def __str__(self):
        return f"SectionZ | Session {self.session_id}"


# ── Class Report ──────────────────────────────────────────────────────
```

```
class ClassReport(models.Model):
    class EstablishmentType(models.TextChoices):
        PUBLIC = "PUBLIC", _("Public")
        PRIVATE = "PRIVATE", _("Privé")

    class StudyLevel(models.TextChoices):
        LEVEL_1 = "1_AS", _("1ère AS")
        LEVEL_2 = "2_AS", _("2ème AS")

    governorate = models.ForeignKey(Governorate,
on_delete=models.CASCADE, related_name="class_reports")
    establishment = models.ForeignKey(
        SchoolEstablishment,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="class_reports"
    )
    establishment_name = models.CharField(max_length=255)
    establishment_type = models.CharField(max_length=10,
choices=EstablishmentType.choices, default=EstablishmentType.PUBLIC)
    study_level = models.CharField(max_length=10,
choices=StudyLevel.choices)
    report_date = models.DateField()

    students_present = models.IntegerField(null=True, blank=True)
    students_refused = models.IntegerField(null=True, blank=True,
default=0)
    students_absent = models.IntegerField(null=True, blank=True,
default=0)

    parental_authorization_required =
models.BooleanField(default=False)
    students_without_authorization = models.IntegerField(default=0)
    questionnaires_collected = models.IntegerField(null=True,
blank=True)

    perturbations = models.IntegerField(choices=[(1, 'Aucune'), (2,
'Quelques élèves'), (3, 'Plusieurs élèves')], default=1)
    serious_work = models.IntegerField(choices=[(1, 'Tous'), (2, 'La
majorité'), (3, 'Moitié ou moins')], default=1)
```

```
    difficulty_level = models.IntegerField(choices=[(1, 'Facile'), (2,
'Moyen'), (3, 'Difficile')], default=1)

    planned_time_minutes = models.IntegerField(null=True, blank=True)
    first_student_time_minutes = models.IntegerField(null=True,
blank=True)
    last_student_time_minutes = models.IntegerField(null=True,
blank=True)

    personal_comments = models.TextField(blank=True, null=True)
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True, related_name="class_reports_created")
    is_finalized = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Report {self.id} | {self.establishment_name} |
{self.report_date}"


# ——— Dynamic Questionnaire Engine
_____________________________________________


class DynamicQuestion(models.Model):
    class QuestionType(models.TextChoices):
        TEXT = "TEXT", _("Text Input")
        RADIO = "RADIO", _("Choice")
        NUMBER = "NUMBER", _("Number Input")
        SECTION = "SECTION", _("New Section")

    code = models.CharField(max_length=50, unique=True,
help_text="Unique identifier (e.g., Z.01 or custom)")
    section = models.CharField(max_length=10, default="Z",
help_text="Section letter where this question belongs")
    label_fr = models.TextField()
    label_ar = models.TextField(blank=True, null=True)
    question_type = models.CharField(max_length=10,
choices=QuestionType.choices, default=QuestionType.TEXT)
    options_json = models.JSONField(default=list, blank=True,
help_text="List of choices for RADIO/choice type: [['val', 'fr', 'ar'],
...]")
```

```
    is_hidden = models.BooleanField(default=False, help_text="If True,
this question (even core ones) will be hidden from the UI")
    is_dynamic = models.BooleanField(default=True, help_text="True if
added by admin, False if it's an override of a core question")
    order = models.IntegerField(default=100)

    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['section', 'order']

    def __str__(self):
        return f"[{self.section}] {self.code}: {self.label_fr[:30]}"
```